
Executive Bored Room


BABIES
Mini
Dognuts
$2.25

Data Preparation Session 8 Handout
String manipulation

The stringr library (part of tidyverse) is designed to make string manipulation easier.

Let's see a very concrete example, what can be wrong, and why do we need string manipulations. We load inspections dataset:

```
library(tidyverse)
library(stringr)
names <- c("ID", "DBAName", "AKAName", "License", "FacilityType", "Risk", "Address",
           "City", "State", "ZIP", "InspectionDate", "InspectionType", "Results",
           "Violations", "Latitude","Longitude","Location")
inspections <- read_csv('inspections.csv', col_names=names, skip=1)
regional_inspections <- unite(inspections, Region, City, State, sep=", ", remove=FALSE)
```

Using a string manipulation function (str_to_upper) we handle the uppercase/lowercase issues by converting everything to uppercase:

```
regional_inspections <- regional_inspections %>% mutate(Region=str_to_upper(Region))
```

We check the region names

```
unique(regional_inspections$Region)
```

We collect a few misspellings of Chicago

```
badchicagos <- c('CCHICAGO, IL', 'CHCICAGO, IL', 'CHICAGOCHICAGO, IL', 'CHCHICAGO, IL',
'CHICAGOI, IL')

regional_inspections <- regional_inspections %>%
  mutate(Region=ifelse(Region %in% badchicagos, 'CHICAGO, IL', Region))
```

We can cross-check what happened:

```
unique(regional_inspections$Region)
```

There are some "CHICAGO, NA" values that we can clearly correct to "CHICAGO, IL"

```
regional_inspections <- regional_inspections %>%
  mutate(Region=ifelse(Region=='CHICAGO, NA', 'CHICAGO, IL', Region))
```

But we don't know what to do with "NA, IL", "NA, NA", or "INACTIVE, IL", so let's set those to missing values

```
nachicagos <- c('NA, IL', 'NA, NA', 'INACTIVE, IL')

regional_inspections <- regional_inspections %>%
  mutate(Region=ifelse(Region %in% nachicagos, NA, Region))
```

A final look:

```
unique(regional_inspections$Region)
```

Putting strings together with stringr

The c is short for concatenate, a function that works like paste()

```
str_c()

str_c("Beautiful","day", sep=" ")
## [1] "Beautiful day"

str_c("Beautiful",NA, sep=" ")
## [1] NA

paste("Beautiful",NA, sep=" ") #base R
## [1] "Beautiful NA"
```

Length

In the base library:

```
nchar(c("Bruce", "Wayne"))
## [1] 5 5

str_length(c("Bruce", "Wayne"))
## [1] 5 5

#factors
f<-factor(c("good","good", "moderate", "bad"))
nchar(f) #nchar throws error
## Error in nchar(f): 'nchar()' requires a character vector

str_length(f)
## [1] 4 4 8 3
```

Extracting substrings

The str_sub(string, start, end) extracts parts of strings based on their location. Its first argument is a string or a vector of strings, the arguments “start” and “end” specify the boundaries of the piece to extract in characters. Both start and end can be negative integers, in which case, they count from the end of the string.

```
str_sub(c("Bruce", "Wayne"), 1, 4)
## [1] "Bruc" "Wayn"

str_sub(c("Bruce", "Wayne"), -3, -1)
## [1] "uce" "yne"
```

Matches

str_detect(string, pattern): answers the question: Does the string contain the pattern?
str_subset(string, pattern): subsetting strings based on match
str_count(string, pattern): counting matches

```
pizzas <- c("cheese", "pepperoni", "sausage and green peppers")

str_detect(pizzas, pattern = "pepper")
## [1] FALSE TRUE TRUE

str_subset(pizzas, pattern = "pepper")
## [1] "pepperoni" "sausage and green peppers"

str_count(pizzas, pattern = "pepper")
## [1] 0 1 1

str_count(pizzas, pattern = "e")
## [1] 3 2 5
```

In pattern, fixed() make a literal comparison (wild characters literally understood). See more at Regex.

Let's go back to our "inspections" dataset. We check most inspected restaurants

```
inspections %>%
  group_by(DBAName) %>%
  summarize(inspections=n()) %>%
  arrange(desc(inspections))
```

Using str_detect, we can find pattern. Like MCDO:

```
inspections <- inspections %>%
  mutate(Mcdo=str_detect(inspections$DBAName, pattern="MCDO"))
sum(inspections$Mcdo)
```

The list of wrongly spelled McDo

```
wrongMcDo <- unique(str_subset(inspections$DBAName, pattern="MCDO"))
wrongMcDo
```

Parsing strings into variables

str_split(): pull apart raw string data into more useful variables. Attention, it creates a list.

```
date_ranges <- c("23.01.2017 - 29.01.2017", "30.01.2017 - 06.02.2017")
split_dates <- str_split(date_ranges, pattern = fixed(" - "))
split_dates
## [[1]]
## [1] "23.01.2017" "29.01.2017"
##
## [[2]]
## [1] "30.01.2017" "06.02.2017"
```

Replacing matches in strings

```
ids <- c("ID#: 192", "ID#: 118", "ID#: 001")

# Replace "ID#: " with "" (nothing)
id_nums <- str_replace(ids, "ID#: ", "")
id_nums
## [1] "192" "118" "001"

phone_numbers <- c("510-555-0123", "541-555-0167")
str_replace_all(phone_numbers, "-", ".")
## [1] "510.555.0123" "541.555.0167"
```

Regular expressions

Regular expressions are the most common way when working with strings. Helpful for extracting something from something.
Which number is the following string: "10202"?
Is there a number in a string: "102a"? What about in this "1O2"?
We would like to separate the following string into 3 columns "2,32.1,0.4"
A special language to describe patterns, fortunately universal, what you learn here, can be applied in any programming language.

grep - global regex print

Is there a pattern in a string? grep(pattern, string)

```
string <- "car"
pattern <- "car"
grep(pattern, string)
## [1] 1

string <- c("car", "cars", "in a car", "truck", "car's trunk")
pattern <- "car"
grep(pattern, string)
## [1] 1 2 3 5
```

grepl

grepl - returns logical value

```
string <- c("car", "cars", "in a car", "truck", "car's trunk")
pattern <- "car"
grepl(pattern, string)
## [1]  TRUE  TRUE  TRUE FALSE  TRUE
```

Meta and special characters
special characters: . \ | ( ) [ ] { } ^ $ * + ?
\- escape character
. - any (just one) character
^ - beginning of a string
$ - end of string

```
string <- c("car", "cars", "in a car", "truck", "car's trunk")
grep("^c.r",string)
## [1] 1 2 5
grep("^c..$",string)
## [1] 1
grep("^c.r.$",string)
## [1] 2
```

Alphanumeric character / Non
w / W

```
grep("\\w", c(" ", "a", "1", "A", "%", "\t"))
## [1] 2 3 4

grepl("\\w", c(" ", "a", "1", "A", "%", "\t"))
## [1] FALSE  TRUE  TRUE  TRUE FALSE FALSE

grep("\\W", c(" ", "a", "1", "A", "%", "\t"))
## [1] 1 5 6

grepl("\\W", c(" ", "a", "1", "A", "%", "\t"))
## [1]  TRUE FALSE FALSE FALSE  TRUE  TRUE
```

Whitespace / Non
s / S

```
grep("\\s", c(" ", "a", "1", "A", "%", "\t"))
## [1] 1 6

grep("\\S", c(" ", "a", "1", "A", "%", "\t"))
## [1] 2 3 4 5
```

Digit / Non
d / D

```
grep("\\d", c(" ", "a", "1", "A", "%", "\t"))
## [1] 3

grep("\\D", c(" ", "a", "1", "A", "%", "\t"))
## [1] 1 2 4 5 6
```

Possible values for a character
using []

```
grep("^[abc]\\w\\w", c("car", "bus", "no", "cars"))
## [1] 1 2 4

grep("^[abc]\\w\\w$", c("car", "bus", "no", "cars"))
## [1] 1 2

grep("^[a-z][a-z][a-z]$", c("Car", "Cars", "cars","car", "no", "three:", "tic", "tac"))
## [1] 4 7 8
```

One or two digits anywhere Exactly one or two digits
| - or
() - group

```
grep("((\\d)|([1-9]\\d))", c("1", "20", "0", "zero", "it is 100%", "09"))
## [1] 1 2 3 5 6
grep("^((\\d)|([1-9]\\d))$", c("1", "20", "0", "nid", "to the 100%", "09"))
## [1] 1 2 3
grep("^((\\d)|(\\d\\d))$", c("1", "20", "0", "nid", "to the 100%", "09"))
## [1] 1 2 3 6
```

Repeating
repeating operators refer to last character or group
? - matches at most 1 times
* - matches at least 0 times
+ - matches at least 1 times
{m} – matches exactly m times
{m, n} – matches between m and n times
{m, } – matches at least m times

```
string <- c("a", "ab", "acb", "accb", "acccb", "accccb")
grep("ac*b", string)
## [1] 2 3 4 5 6
grep("ac+b", string)
## [1] 3 4 5 6

string <- c("a", "ab", "acb", "accb", "acccb", "accccb")
grep("ac?b", string, value=TRUE)
## [1] "ab" "acb"
grep("ac{2}b", string, value = TRUE)
## [1] "accb"
grep("ac{2,}b", string, value = TRUE)
## [1] "accb" "acccb" "accccb"
grep("ac{2,3}b", string, value = TRUE)
## [1] "accb" "acccb"
```

All lowercase letter words

```
grep("^([a-z]+ )*[a-z]+$", c("words", "words or sentences", "123 no", "Words"," word","word 123"))
## [1] 1 2
```

Words with lowercase letters and length from 3-5

```
grep("^[a-z]{3,5}$", c("words", "words or sentences", "123 no", "Words"," word","word 123","hey"))
## [1] 1 7
```

Signed numbers

```
grep("^[+-]?(0|[1-9][0-9]*)$", c("++0", "+1", "01", "-99"))
## [1] 2 4
```

Greedy and Lazy Repetition

the repetition operators or quantifiers are greedy
how to make them lazy?
regexpr
regmatches

```
string<-"This is a <EM>first</EM> test"
pattern<-"<.+>"
r<-regexpr(pattern,string)
regmatches(string, r)
## [1] "<EM>first</EM>"

##lazy
pattern<-"<.+?>"
r<-regexpr(pattern,string)
regmatches(string, r)
## [1] "<EM>"
```

Sub-pattern (group catching)

(….(..(.))..(..(.)…(.))
“naming” goes from left to right

```
string <- c("(1,2)", "( -2, 7)", "( -3 , 45)", "(a, 3)")
pattern<-"\\(\\s*([+-]?(0|[1-9][0-9]*))\\s*,\\s*([+-]?(0|[1-9][0-9]*))\\s*\\)"
lidx <- !grepl(pattern, string)
komp1 <- sub(pattern, "\\1", string)
komp1[lidx] <- NA
komp2 <- sub(pattern, "\\3", string)
komp2[lidx] <- NA
as.integer(komp1)
## [1] 1 -2 -3 NA
as.integer(komp2)
## [1] 2 7 45 NA
```

gregexpr
Finds all positions and lengths of matched patterns, and returns a list.

```
text<-"Yesterday I had 100 Euros, today I only have 45 Euros left."
gregexpr("(\\d+)", text)
## [[1]]
## [1] 17 46
## attr(,"match.length")
## [1] 3 2
## attr(,"index.type")
## [1] "chars"
## attr(,"useBytes")
## [1] TRUE
```

regmatches
Extracts or replaces matched substrings from match data obtained by regexpr, gregexpr or regexec.

```
text<-"Yesterday I had 100 Euros, today I only have 45 Euros left."
regmatches(text, gregexpr("\\d+",text))
## [[1]]
## [1] "100" "45"
```

Stringr and regular expressions
str_subset (grep)

```
fruit%>%str_subset("\\s")
## [1] "bell pepper" "blood orange" "canary melon"
## [4] "chili pepper" "goji berry" "kiwi fruit"
## [7] "purple mangosteen" "rock melon" "salal berry"
## [10] "star fruit" "ugli fruit"
```

str_detect (grepl)
str_extract/str_extract_all (gregexpr+regmatches)

```
text<-"Yesterday I had 100 Euros, today I only have 45 Euros left.“
str_extract(text,"\\d+")
## [1] "100"
str_extract_all(text,"\\d+")
## [[1]]
## [1] "100" "45"
```

We go back to inpatient to find alternate spellings of McDonalds

```
inspections %>%
  filter(grepl("McDo", DBAName, ignore.case=TRUE)) %>%
  select(DBAName) %>%
  unique() %>% View()
```

There is a restaurant that is not McDonald's:

```
alternates <- inspections %>%
  filter(grepl("McDo", DBAName, ignore.case=TRUE)) %>%
  filter(DBAName!='SARAH MCDONALD STEELE') %>%
  select(DBAName) %>%
  unique() %>%
  pull(DBAName)

alternates
```

We replace them all with MCDONALDS.

```
inspections <- inspections %>%
  mutate(DBAName=ifelse(DBAName %in% alternates, 'MCDONALDS', DBAName))
```

We check most inspected restaurants again.

```
inspections %>%
  group_by(DBAName) %>%
  summarize(inspections=n()) %>%
  arrange(desc(inspections))
```

What a difference!